

We have completed the code review and basic functional testing of the Rust and Solidity components in Bifrost, and have not identified any significant security vulnerabilities so far. However, there are still a few questions we would like to raise. Furthermore, we will proceed with more in-depth testing next.

Rust Part

vtoken-minting-1 get_currency_amount_by_v_currency_amount function logic problem

- **Lines:** pallets/vtoken-minting/src/impls.rs #L1204
- **Risk Level:** Low
- **Description:** In the get_currency_amount_by_v_currency_amount function, the corresponding currency_amount is calculated based on the vToken supply. However, if the vToken issuance is 0, the function will simply return v_currency_amount. Although this scenario seems unlikely, if an administrator sets v_currency_issuance to 0, users would be unable to redeem, as modifying v_currency_issuance could trigger an underflow during subtraction.

```
BalanceOf<T>,  
>>::get_token_pool(currency_id);  
let v_currency_issuance = Self::get_v_currency_issuance_inner(v_currency_id)?;  
  
if BalanceOf::<T>::zero().eq(&v_currency_issuance) {  
    Ok(v_currency_amount)  
} else {  
    Ok(multiply_by_rational_with_rounding(  

```

- **Recommendation:** Would it be more appropriate to handle this case by returning a currency_amount of 0 when v_currency_issuance is 0?

vtoken-minting-2 Key functions are missing events

- **Risk Level:** Info
- **Description:** Several operations in this function lack event emissions. For example, functions such as update_token_pool, update_unlocking_total, update_token_unlock_ledger, mint_without_transfer, and process_redeem do not emit events when state changes occur.

- **Recommendation :** It is recommended to declare events for these critical operations and trigger them within the corresponding functions, so that on-chain state changes can be observed and tracked externally.

slp-1 Lack of overflow checks

- **Lines:** pallets/slp-v2/src/ethereum_staking/types.rs #L62-71
- **Risk Level:** Low
- **Description :** As shown in the figure, the function uses `saturating_accrue` and `saturating_reduce`. In particular, `saturating_reduce` may still pose a potential overflow risk..

```
/// Adds the specified amount to the total locked amount.
pub fn add_lock_amount(&mut self, amount: Balance) {
    self.locked.saturating_accrue(amount);
}

/// Subtracts the specified amount of the total locked amount.
pub fn subtract_lock_amount(&mut self, amount: Balance) {
    self.locked.saturating_reduce(amount);
}
```

- **Recommendation :** It is recommended to add additional checks or constraints when using `saturating_reduce`, for example, ensuring the minuend is greater than or equal to the subtrahend before the operation, or designing the logic to avoid extreme cases altogether. Alternatively, consider using `checked_sub` or other safe subtraction methods, and return an error if an overflow occurs.

slp-2 Inaccurate transfer amount

- **Lines:** pallets/slp-v2/src/common/impls.rs #L149, 195
- **Risk Level:** Medium
- **Description :** In the `do_transfer_to` function, if the staking type is `AstarDappStaking`, the function uses the entire balance of the entrance account as the transfer amount instead of the amount specified in the parameter. This behavior appears to be incorrect. Additionally, the event should record amount rather than the full balance.

```

let entrance_account_free_balance =
  T::MultiCurrency::free_balance(currency_id, &entrance_account);

match staking_protocol {
  StakingProtocol::AstarDappStaking => {
    let dest_beneficiary_location = staking_protocol
      .get_dest_beneficiary_location::<T>(delegator.clone())
      .ok_or(Error::<T>::UnsupportedStakingProtocol)?;
    T::XcmTransfer::transfer(
      entrance_account.clone(),
      currency_id,
      entrance_account_free_balance,
      dest_beneficiary_location,
      WeightLimit::Unlimited,
    )
    .map_err(|_| Error::<T>::DerivativeAccountIdFailed)?;
  }
  StakingProtocol::EthereumStaking => {

```

- **Recommendation :** It is recommended to add additional checks or constraints when using saturating_reduce, for example, ensuring the minuend is greater than or equal to the subtrahend before the operation, or designing the logic to avoid extreme cases altogether. Alternatively, consider using checked_sub or other safe subtraction methods, and return an error if an overflow occurs.

Other Questions

(1) In the vtoken-minting module, the calculate_incentive_vtoken_amount function uses the square root of the percentage to calculate the reward share, aiming to prevent concentration by large holders. However, it should be noted that this approach may cause the total percentage to exceed 100%, meaning the cumulative share of all users can surpass the total allocation. It is necessary to verify whether this behavior aligns with the business design and intended logic.

(2) In the slp-v2 module, the update_token_exchange_rate function increases the issuance of vTokens based on pool_value, but no corresponding inflow of the underlying currency is observed. It is necessary to verify whether this approach is reasonable and aligns with the business logic and asset balance principles.

(3) We noticed that multiple StakingProtocols are declared under the types module, including MoonriverParachainStaking, AstarDappStaking, KusamaStaking, MoonbeamParachainStaking, PolkadotStaking, and EthereumStaking. However, in the

do_transfer_to function, only AstarDappStaking and EthereumStaking are allowed. It is necessary to verify whether this restriction aligns with the project's business requirements.

```
match staking_protocol {
  StakingProtocol::AstarDappStaking => {
    let dest_beneficiary_location = staking_protocol
      .get_dest_beneficiary_location::::UnsupportedStakingProtocol)?;
    T::XcmTransfer::transfer(
      entrance_account.clone(),
      currency_id,
      entrance_account_free_balance,
      dest_beneficiary_location,
      WeightLimit::Unlimited,
    )
    .map_err(|_| Error::::DerivativeAccountIdFailed)?;
  }
  StakingProtocol::EthereumStaking => {
    let (amount, to, dest, payer, fee) = if let (
      Some(amount),
      Delegator::Ethereum(to)
    ) {
      (amount, to, dest, payer, fee)
    } else {
      return Err(Error::::UnsupportedStakingProtocol);
    }
    T::XcmTransfer::transfer(
      entrance_account.clone(),
      currency_id,
      entrance_account_free_balance,
      dest,
      WeightLimit::Unlimited,
    )
    .map_err(|_| Error::::DerivativeAccountIdFailed)?;
  }
}
```

(4) In the transfer_to_by_redeem_type function, if the amount being redeemed by the user is too small (less than ed), no actual transfer is performed; instead, only the ledger is updated.

```
/// - ~^(BalanceOf<T>, RedeemTo<T::AccountId>): The redeem currency amount, redeem to.
pub fn transfer_to_by_redeem_type(
  redeemer: T::AccountId,
  redeem_currency_id: CurrencyId,
  mut redeem_currency_amount: BalanceOf<T>,
  entrance_account_balance: BalanceOf<T>,
  redeem_type: RedeemType<T::AccountId>,
) -> Result<BalanceOf<T>, RedeemTo<T::AccountId>, DispatchError> {
  let entrance_account = T::EntranceAccount::get().into_account_truncating();
  if entrance_account_balance >= redeem_currency_amount {
    if let RedeemType::Native = redeem_type {
      let ed = T::MultiCurrency::minimum_balance(redeem_currency_id);
      if redeem_currency_amount >= ed {
        T::MultiCurrency::transfer(
          redeem_currency_id,
          &entrance_account,
          &redeemer,
          redeem_currency_amount,
          ExistenceRequirement::AllowDeath,
        );
      }
      return Ok((redeem_currency_amount, RedeemTo::Native(redeemer)));
    }
    if let RedeemType::HyperBridge(dest, to) = redeem_type {

```

(5) In the vtoken-minting module, the handle_ledger_by_currency function, when processing redeem, increments MinTimeUnit by 1 if the current MinTimeUnit has been processed but ongoing_time + unlock_duration is still greater than MinTimeUnit. However, since ongoing_time is updated by another module, slp-v2, there may be delays in updating, which can cause MinTimeUnit to accumulate up to ongoing_time +

unlock_duration. In this scenario, if a user submits a redeem request, the unlock time could be directly set to ongoing_time + unlock_duration, potentially allowing the user to bypass the normal waiting period and unlock immediately.

(6) In the vtoken-minting module, within the calculate_incentive_vtoken_amount function, users can claim rewards based on their share of veBNC tokens. However, the function calculates the total amount using the BbBNC token supply. It should be confirmed whether this design aligns with the intended business logic.

```
T::BlockNumberProvider::current_block_number();
// get the veBNC total amount
let vebnc_total_issuance = T::BbBNC::total_supply(Some(current_block_number))
    .map_err(|_| Error::<T>::VeBNCCheckingError)?;
ensure!(
    vebnc_total_issuance > BalanceOf::<T>::zero(),
    Error::<T>::BalanceZero
);

// get the veBNC balance of the minter
let minter_vebnc_balance =
    T::BbBNC::balance_of(minter, None).map_err(|_| Error::<T>::VeBNCCheckingError)?;
ensure!(
    minter_vebnc_balance > BalanceOf::<T>::zero(),
    Error::<T>::NotEnoughBalance
);

// get the percentage of the veBNC balance of the minter to the total veBNC amount and
// get the square root of the percentage
let percentage = Permill::from_rational(minter_vebnc_balance, vebnc_total_issuance);
let sqrt_percentage =
    FixedU128::from_inner((percentage * 1_000_000_000_000_000u128).sqrt());
let percentage =
    Permill::from_rational(sqrt_percentage.into_inner(), 1_000_000_000_000_000u128);
// get the total issuance of the vtoken for rate calculation
let v_currency_issuance = Self::get_v_currency_issuance_inner(v_currency_id)?;
```

Solidity Part

bifrost-io-1 asyncMint Function Sets the redeem Parameter to true

- **Lines:** VTokenBase.sol #L 224
- **Risk Level:** Low
- **Description :** The asyncMint function invokes the teleport method of the teTokenGateway contract, but the redeem parameter is set to true, identical to the setting in asyncRedeem. As a result, despite the distinct function names ("Mint" vs. "Redeem"), the underlying logic effectively relies on the same redeem pathway. If

tokenGateway processes the call strictly in redeem mode, this could blur the functional distinction between asyncMint and asyncRedeem, leading to functional ambiguity and potential logic risks.

```
216
217     function asyncMint(uint256 relayerFee, uint64 timeout) external payable onlyTriggerAddress {
218         bytes memory data = abi.encode(uint32(block.chainid), currentCycleMintVTokenAmount);
219         bytes32 assetId = keccak256(bytes(ERC20(asset()).symbol()));
220         TeleportParams memory params = TeleportParams({
221             amount: currentCycleMintTokenAmount,
222             relayerFee: relayerFee,
223             assetId: assetId,
224             redeem: true,
225             to: BIFROST_TOKEN_GATEWAY,
226             dest: bifrostDest,
227             timeout: timeout,
228             nativeCost: 0,
229             data: data
230         });
231         tokenGateway.teleport(params);
232     }
```

- **Recommendation:** It is recommended to verify whether the redeem parameter value is set appropriately to align with the intended function design, preventing potential logic ambiguity or functional inconsistencies.

bifrost-io-2 Centralization Risk

- **Lines:** VTokenBase.sol #L 455
- **Risk Level:** Low
- **Description:** The contract implements a burn function that allows the admin to burn VTokens from any account. This grants the administrator unilateral authority over user assets, introducing a centralization risk. If the admin's private key is compromised or abused, user funds could be affected, undermining the decentralization and trust assumptions of the system.

```
448     }
449
450     // ===== ERC6160 functions =====
451     function mint(address _to, uint256 _amount) public onlyRoleAdmin {
452         super._mint(_to, _amount);
453     }
454
455     function burn(address _from, uint256 _amount) public onlyRoleAdmin {
456         super._burn(_from, _amount);
457     }
458
459     function supportsInterface(bytes4 interfaceId) public view virtual override
460     return interfaceId == type(ERC4626).interfaceId || interfaceId == type
461         || super.supportsInterface(interfaceId);
```

- **Recommendation:** It is recommended to manage the relevant permissions via a

multi-signature wallet. This ensures that sensitive operations require approval from multiple independent keys, mitigating single point of failure and reducing centralization risks.

bifrost-io-3 Potential Locking Risk of Platform Token

- **Lines:** ValidatorManager.sol #L 82
- **Risk Level:** Low
- **Description :** The batchConsolidation function validates whether the provided msg.value meets the required cost; however, any excess amount is not refunded and remains locked in the contract. This results in users' overpaid funds being stranded and irretrievable, leading to potential value loss.

```
68
69     function batchConsolidation(bytes[] calldata sourcePubkeys, bytes calldata targetPubkey
70         external
71         payable
72         onlyAuthority(msg.sender)
73     {
74         uint256 batchSize = sourcePubkeys.length;
75         require(batchSize >= MIN_VALIDATORS, MinimumValidatorRequired());
76         require(batchSize <= MAX_SOURCE_VALIDATORS, TooManySourceValidators());
77         if (targetPubkey.length != VALIDATOR_PUBKEY_LENGTH) {
78             revert InvalidTargetPubkeyLength(targetPubkey);
79         }
80
81         uint256 consolidationFee = getConsolidationFee();
82         require(msg.value >= batchSize * consolidationFee, InsufficientFeePerValidator());
83
84         for (uint256 i = 0; i < batchSize; ++i) {
85             if (sourcePubkeys[i].length != VALIDATOR_PUBKEY_LENGTH) {
```

- **Recommendation:** It is recommended to enforce that the amount of msg.value provided in the batchConsolidation function must exactly match the required fee, thereby preventing excess funds. Alternatively, consider implementing a withdrawal function that allows the admin or users to reclaim any excess funds, ensuring they are not permanently locked in the contract.

Other Questions

(1) The conversion between VToken and the token in the contract is performed through an Oracle, meaning that the tokenAmount and vTokenAmount are updated only when onAccept is called. Therefore, the exchange rate remains fixed for a certain period. How, then, is the fee reflected as the remaining VToken earnings for users?

```

358
359     function _convertToShares(uint256 assets, Math.Rounding rounding)
360     {
361         internal
362         view
363         virtual
364         override
365         returns (uint256)
366     {
367         return oracle.getVTokenAmountByToken(address(asset()), assets, rounding);
368     }
369
370     function _convertToAssets(uint256 shares, Math.Rounding rounding)
371     {
372         internal
373         view
374         virtual
375         override
376         returns (uint256)
377     {
378         return oracle.getTokenAmountByToken(address(asset()), shares, rounding);
379     }

```

```

/// @notice Accept the post request
/// @param incoming The incoming post request
/// @dev The post request is from Bifrost
function onAccept(IncomingPostRequest calldata incoming)
{
    external
    override
    onlyIsmpHost
    onlyFromBifrost(incoming.request)
    {
        bytes memory body = incoming.request.body;
        (address token, uint256 tokenAmount, uint256 vtokenAmount) = abi.decode(body, (address, uint256, uint256));
        poolInfo[token].tokenAmount = tokenAmount;
        poolInfo[token].vTokenAmount = vtokenAmount;
        emit SetTokenAmount(token, tokenAmount, vtokenAmount);
    }
}

```